Page 1: Password Security Policy

Section 1: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 2: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 3: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 4: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 5: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 6: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 7: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 8: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 9: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 10: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 11: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 12: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 13: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Section 14: Password Security Policy
Employees must reset their passwords every 90 days.
Every password must contain at least 12 characters.
Passwords should include uppercase letters, lowercase
letters, numbers, and special characters.

Employees must never share their login credentials.
Multi-factor authentication is mandatory for all
company accounts. The IT department provides password
reset support and helps employees recover locked accounts.

Suspicious login activity must be reported immediately.
Employees should avoid saving passwords on public computers.
The security team reviews access logs regularly.

Page 2: Employee Leave Policy

Section 1: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 2: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 3: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 4: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 5: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 6: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 7: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 8: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 9: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 10: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 11: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 12: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 13: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Section 14: Employee Leave Policy
Employees receive 20 days of paid leave annually.
Leave requests must be submitted to the reporting manager.
Managers review requests based on team availability.

Employees should submit planned leave requests at least
seven days before the intended leave date.
Emergency leave must be communicated as soon as possible.

The human resources department maintains leave records.
Unused leave is handled according to company policy.
Employees can check their remaining leave balance
through the internal employee portal.

Page 3: Attendance Management

Section 1: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 2: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 3: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 4: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 5: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 6: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 7: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 8: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 9: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 10: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 11: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 12: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 13: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Section 14: Attendance Management
Employees must record their attendance every working day.
The normal office working hours are 9 AM to 6 PM.
Employees should inform their manager about late arrivals.

Attendance records are maintained by the HR department.
Missing attendance entries should be corrected through
the attendance management portal.

Managers review attendance reports every month.
Repeated attendance issues may require a discussion
with the employee and the HR team.

Page 4: Document Upload System

Section 1: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 2: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 3: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 4: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 5: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 6: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 7: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 8: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 9: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 10: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 11: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 12: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 13: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Section 14: Document Upload System
The document search application supports PDF, DOCX, and TXT file formats. Users upload documents through the document management interface.

After upload, the system identifies the file format and extracts its readable text. PDF files are processed page by page. Word files are processed paragraph by paragraph. Text files are read directly.

Unsupported file formats are rejected with an error. The original filename is stored as source metadata. Uploaded documents are prepared for text chunking.

Page 5: Document Chunking

Section 1: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 2: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 3: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 4: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 5: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 6: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 7: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 8: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 9: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 10: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 11: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 12: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 13: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Section 14: Document Chunking
Document chunking divides long text into smaller sections. Each section is called a chunk. Chunking helps the search system retrieve relevant information efficiently.

Fixed-size chunking uses a defined token limit. Sentence-based chunking tries to preserve complete sentences. Recursive character splitting uses separators such as paragraph breaks, line breaks, and spaces.

Overlapping chunks preserve context between neighboring sections. A common configuration uses 512 characters with an overlap of 50 characters.

Page 6: Embedding Generation

Section 1: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 2: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 3: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 4: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 5: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 6: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 7: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 8: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 9: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 10: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 11: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 12: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 13: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Section 14: Embedding Generation
Embedding models convert text into numerical vectors.
These vectors represent the semantic meaning of text.

The all-MiniLM-L6-v2 model generates 384-dimensional
embeddings. Embedding models can represent both documents
and user queries in the same vector space.

Similar meanings generally produce vectors with higher
cosine similarity. Embeddings are stored alongside
document metadata so the original source can be identified.

The system generates embeddings for every document chunk
before inserting them into the vector database.

Page 7: Semantic Search

Section 1: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 2: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 3: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 4: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 5: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 6: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 7: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 8: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 9: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 10: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 11: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 12: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 13: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Section 14: Semantic Search
Semantic search retrieves information based on meaning
rather than requiring exact keyword matches.

A user submits a natural-language question.
The application converts the question into an embedding.
The query vector is compared with stored document vectors.

The system ranks the closest matches by similarity score.
The highest-ranking chunks are returned to the user.
Results include the source document and page information.

Semantic search can identify related expressions even
when the query and document use different words.

Page 8: Vector Database

Section 1: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 2: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 3: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 4: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 5: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 6: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 7: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 8: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 9: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 10: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 11: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 12: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 13: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Section 14: Vector Database
A vector database stores numerical embeddings and metadata.
It supports similarity search across large collections
of document chunks.

Each stored record contains a vector, a unique identifier,
and a metadata payload. Metadata can include the source,
page number, document type, and section heading.

Approximate nearest neighbor indexes improve search speed.
Metadata filters can restrict results to selected documents,
file types, or page ranges.

The database supports inserting, updating, searching,
and deleting document records.

Page 9: Search API

Section 1: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 2: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 3: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 4: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 5: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 6: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 7: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 8: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 9: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 10: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 11: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 12: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 13: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Section 14: Search API
The document search application exposes a FastAPI service.
The POST /search endpoint accepts a user query and top_k.
The top_k value controls the number of returned results.

The API converts the query into an embedding and searches
the vector database. Each result includes its source,
page number, section, similarity score, and text.

The GET /health endpoint reports application status.
The upload endpoint accepts supported document files.
API authentication protects access to search operations.

Page 10: Application Security

Section 1: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 2: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 3: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 4: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 5: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 6: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 7: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 8: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 9: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 10: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 11: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 12: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 13: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.

Section 14: Application Security
API keys are used to authenticate requests to protected application endpoints. Secret keys must be stored in environment variables rather than committed to Git.

The .env file contains local development credentials. The .gitignore file prevents secret files from being included in version control.

Input validation checks uploaded files and search queries. The application should reject unsupported file types and invalid parameter values.

Error responses should provide useful messages without revealing passwords, API keys, or internal secrets.